CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

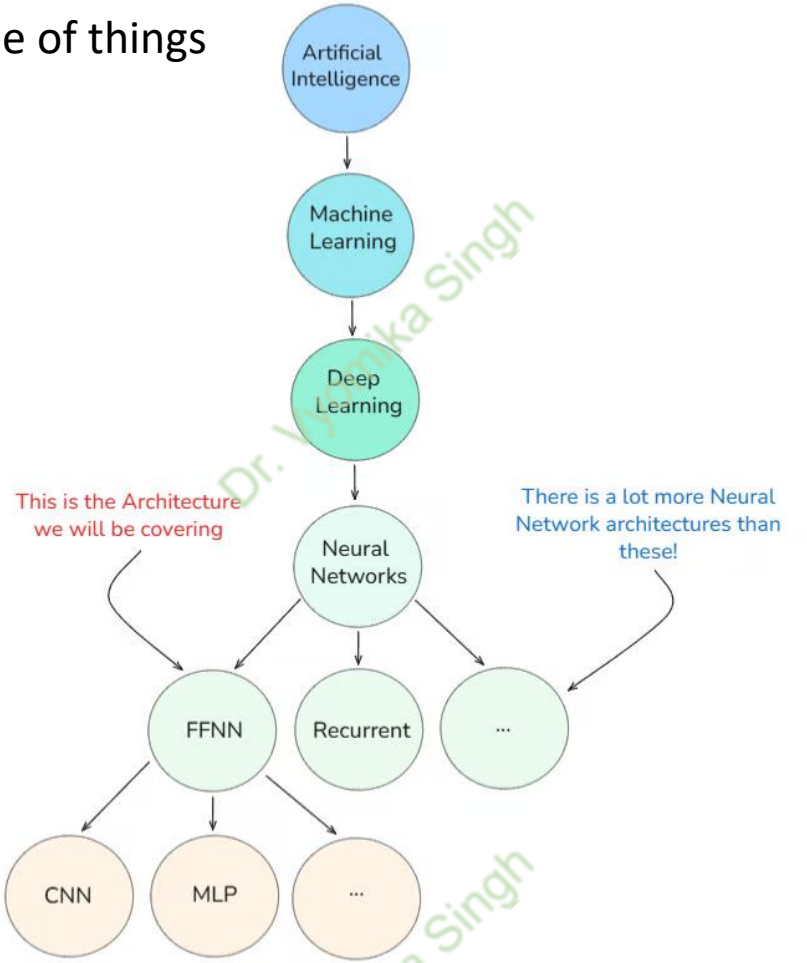
Course Instructor/Faculty: Dr. Vyomika Singh

UNIT 2: Feed Forward Networks (Topics covered in this PPT)

Multilayer Perceptron (MLP) and activation functions. Logistic Regression**, Learning parameters, Loss functions**, Empirical Risk Minimization, Gradient Descent and Backpropagation algorithm. Regularization techniques. Difficulty of training deep neural networks, Greedy layer wise training.

CSN 342
Deep Learning

Recap, w.r.t the broad scheme of things



CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

What is function *f* in deep learning?

In DL, a **function *f*** usually means something like:

$$y = f(x ; \theta)$$

where:

1. x = input (image, text, numbers, etc.)
2. θ = parameters (weights, biases)
3. y = output (prediction)
4. f = the neural network (a big nested composition of functions)

*We train the network by **changing ϑ** so that the output becomes better.*

df/dx means rate of change of f when x changes.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

What is d*f*/d*x* in deep learning?

df/dx means rate of change of f when x changes.

Physical interpretation:

Think of
1. x = position
2. f(x) = height of a hill

Then: d*f*/d*x* is the **slope of the hill** at that point:

1. Positive → uphill
2. Negative → downhill
3. Zero → flat (peak or valley)

So physically: **derivative = sensitivity / slope / local change**

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Partial differentiation, why "partial"

In deep learning, f **almost** never depends on just one variable:

f = f (w1,w2,…,wn)

Example:

Loss L = L (w1,w2,…,wn)

A partial derivative: ∂f/∂wi

How does f change if i changes only with wi, keeping all other variables fixed?

In deep learning this is used in two main concepts:

1. Training = minimizing loss
2. Backpropagation = chain rule of derivatives

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Integration; where it is used in DL

Integration is the **opposite** of differentiation (accumulating instead of measuring change).

Mathematical meaning:

$$\int f(x)dx$$

Means accumulate/sum up values of $f(x)$ over a range

Where is integration used in ML/DL?

Not as central as derivatives, but important in:

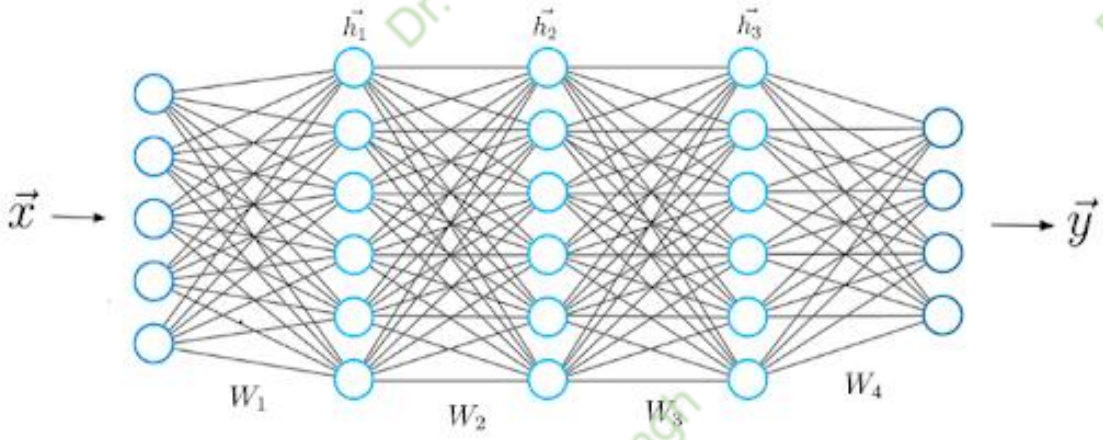
1. Probability
2. Losses as expectations
3. Physics based ML/Continuous modelling

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Summary

1. f = your model or loss function
2. df/dx = sensitivity: "If I change x a bit, how much does output change?"
3. **Partial derivatives** = sensitivity w.r.t. one parameter at a time
4. **Gradients** = collection of all partial derivatives → direction of steepest increase
5. **Gradient descent** = move downhill to reduce error
6. **Integration** = accumulation, averaging, total probability, expected loss



CSN 342
Deep Learning

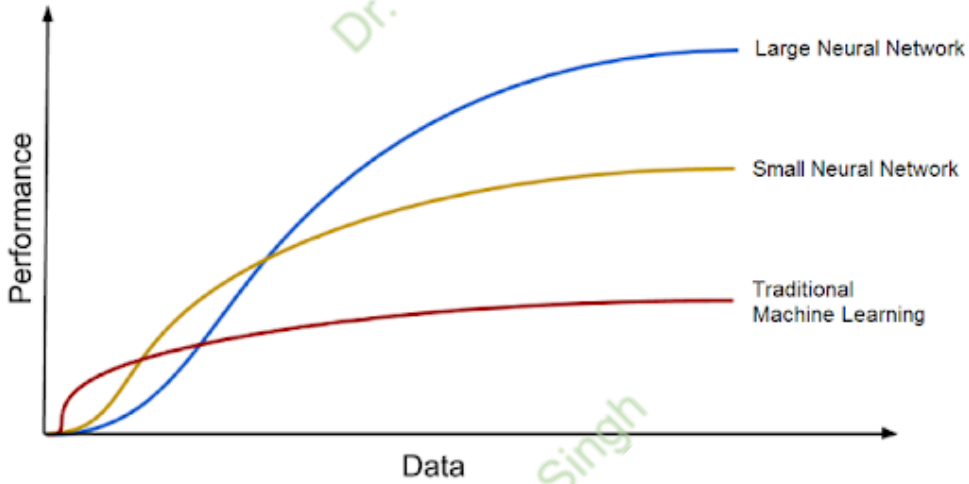
DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Neural Networks

A neural network (NN) is a computational model inspired by the human brain.
It is made of:

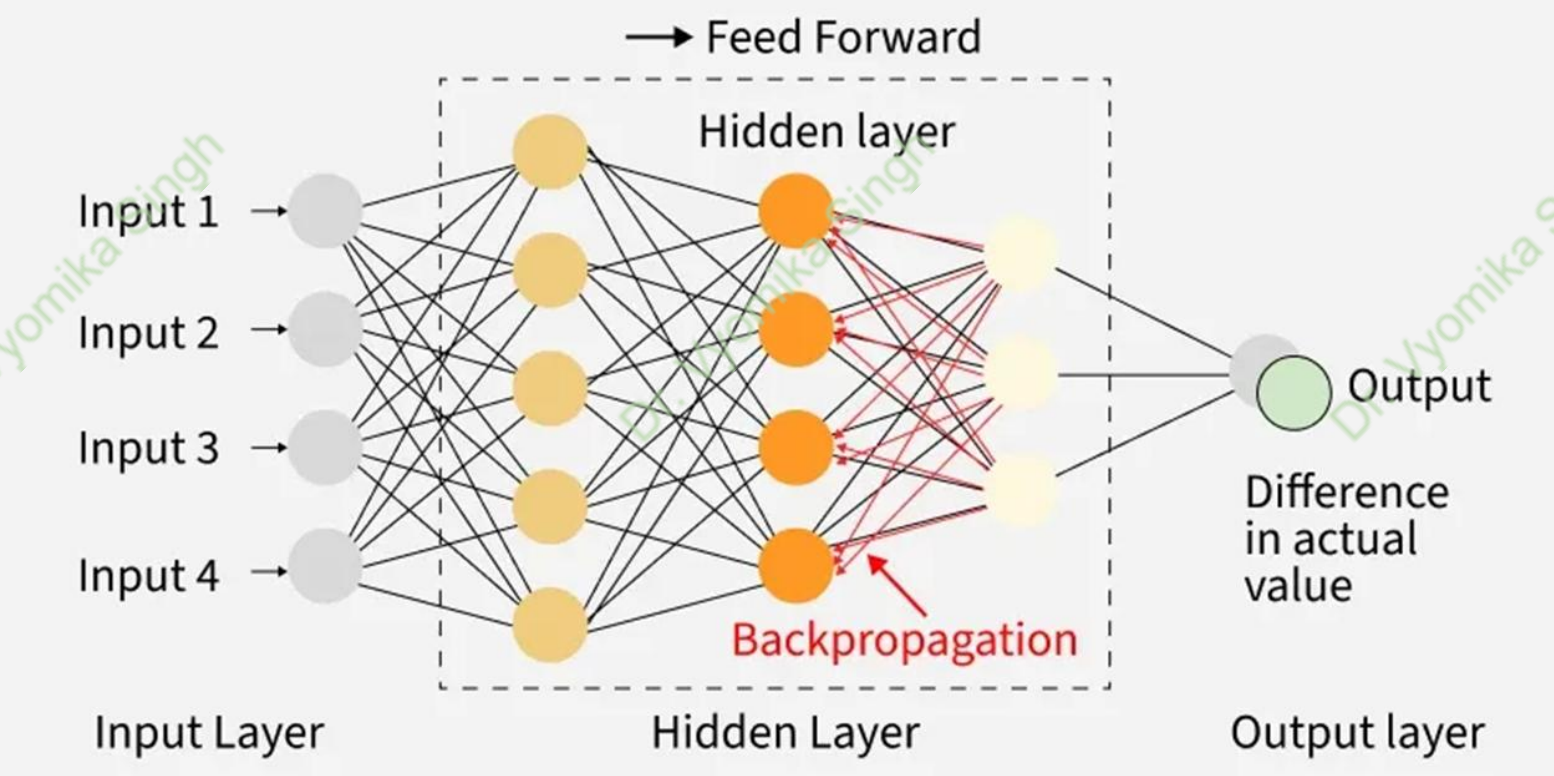
1. **Neurons (nodes):** does calculations
2. **Weights:** decide importance of inputs
3. **Layers:** a.) Input layer-receives data, b.) Hidden layer(s)-processes data and c.) Output layer-gives final result.

Information flows through the network, gets transformed, and produces an output



CSN 342
Deep Learning

Neural Networks



CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks

A Feedforward Neural Network (FNN) is a type of artificial neural network **where information moves only in one direction**, from the input layer through any hidden layers and finally to the output layer. This is the simplest kind of artificial neural network. There are no cycles or loops in the network. The term **"feedforward"** refers to the fact that the connections between the units do not form a cycle, unlike in more complex types of networks (like RNN).

This straightforward design makes them well-suited for tasks that require a simple, one-way processing of data including pattern recognition and predictive modelling.

Examples: Perceptron, MLP, CNNs

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks

In the late 1980s, Feed forward neural network's revival was fueled by **two key developments**:

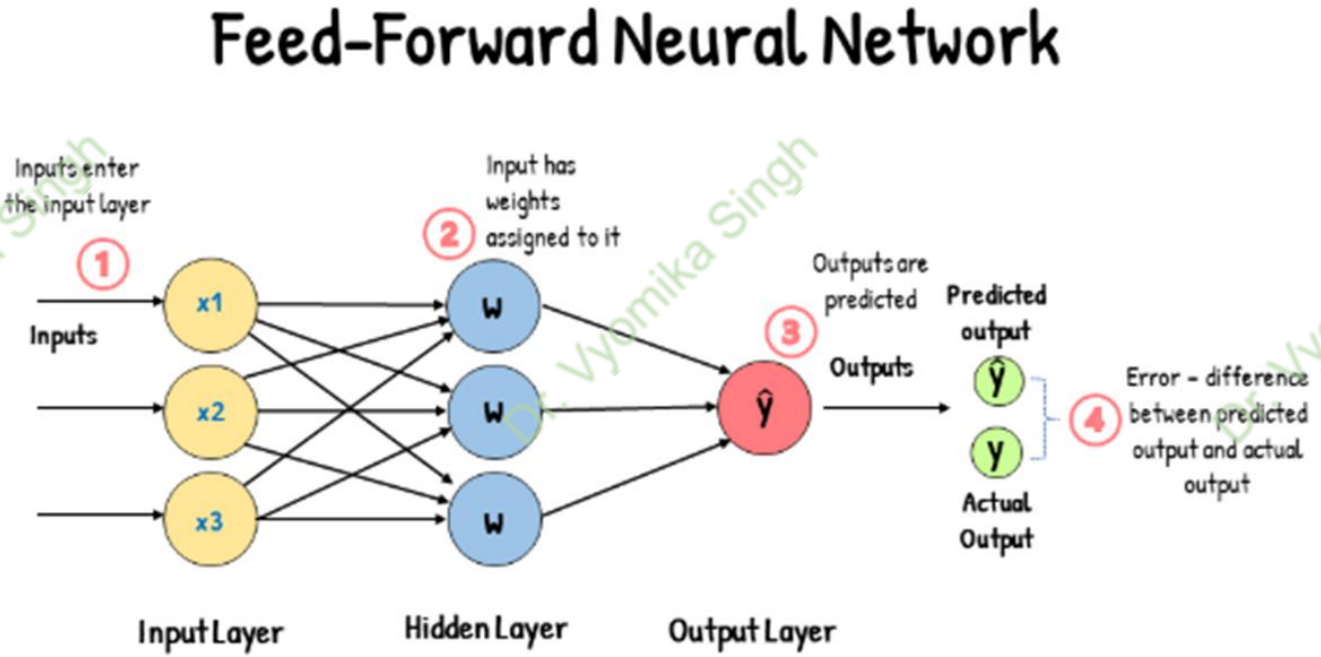
1. ***Multi-Layer Perceptrons (MLPs)***: These networks introduced additional layers of neurons, allowing them to **learn more complex relationships**.

2. ***Backpropagation algorithm*****:** Invented by David Rumelhart, Geoffrey Hinton, and Ronald J. Williams in 1986, Backpropagation provided a **powerful tool for training MLPs by efficiently calculating the gradient of the error function with respect to network weights**.

FNNs represent a fundamental building block of deep learning, that paved the way for more complex and powerful architectures.

CSN 342
Deep Learning

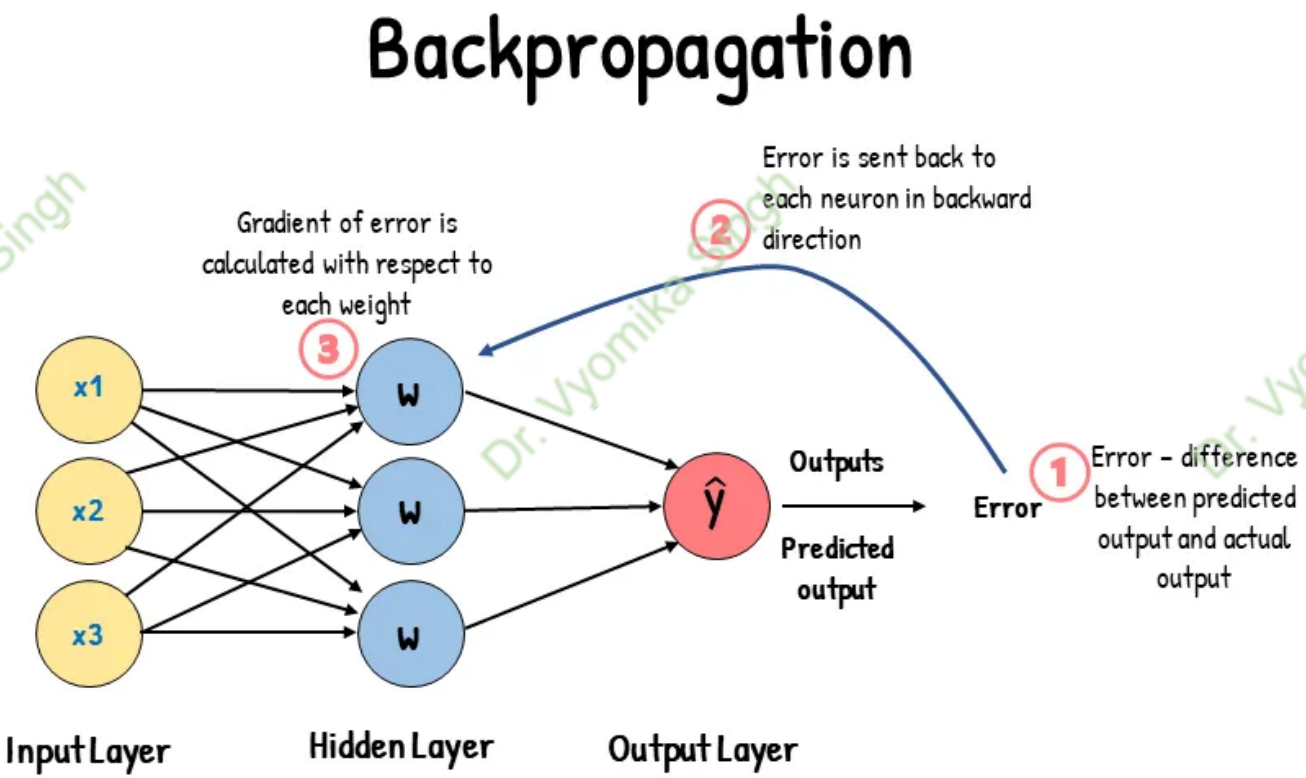
Concept 1:



Feed Forward Neural Networks

CSN 342
Deep Learning

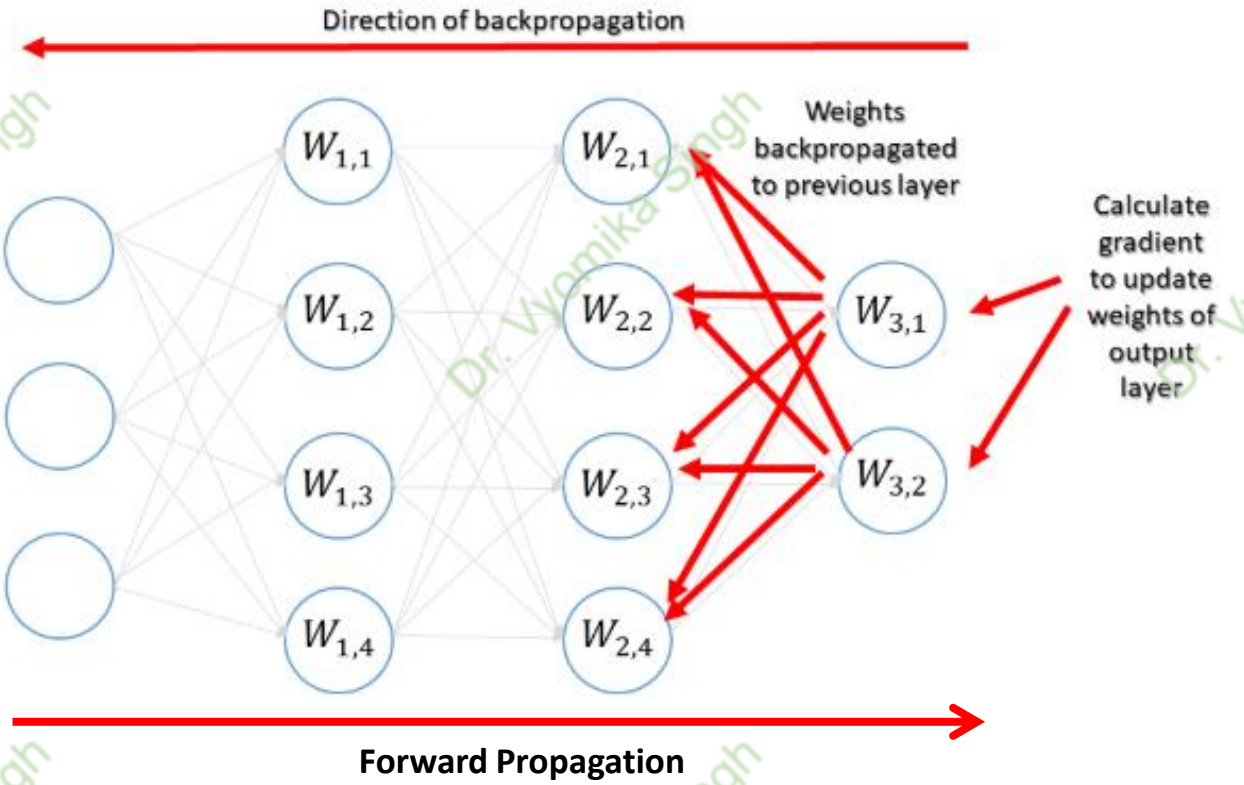
Concept 2:



Feed Forward Neural Networks

CSN 342
Deep Learning

Feed Forward Neural Networks



CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks (Summary)

A **feedforward network** is the **simplest type of neural network**.

1. Information flows **in one direction only**
2. From **input → hidden layer(s) → output**

Common examples

1. Single Layer Perceptron
2. Multilayer Perceptron (MLP)
3. CNNs

Widely used in:

- Image classification
- Spam detection
- Regression problems

Feedforward Neural Networks → Static Neural Networks

(Static = output depends only on current input)

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks

Computational Considerations of Feed-Forward Networks:

1. **Simple Structure:** Feed-forward networks follow a straight path from input to output. This makes them easier to implement and tune.
2. **Parallel Computation:** Inputs can be processed in batches, enabling fast training using modern hardware.
3. **Efficient Backpropagation:** They use standard backpropagation which is stable and well-supported across frameworks.
4. **Lower Resource Use:** No memory of past inputs means less overhead during training and inference.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks: Real world applications

FNNs often serve as building blocks for more complex architectures. While more complex deep learning architectures like convolutional neural networks (CNNs) and recurrent neural networks (RNNs) have emerged, FNNs are still used in real-life implementations.

FNNs remain relevant today due to its simplicity, interpretability and efficiency. FNNs are versatile and can be used for a wide range of tasks, including classification, regression etc.

Some examples of real-world FNN applications:

1. ***Spam filtering***: FNNs are used to classify emails as spam or not spam.
2. ***Fraud detection***: FNNs are used to analyze financial transactions to identify fraudulent activity.
3. ***Medical diagnosis***: FNNs are used to analyze medical images and data to assist in diagnosis
4. ***Credit scoring***: FNNs are used to assess the creditworthiness of individuals and businesses.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feed Forward Neural Networks: Real world applications

5. ***Time series forecasting***: FNNs can predict future values of time series data, such as stock prices and energy consumption.
6. ***eCommerce***: FNNs are used in recommendation systems that suggest products to customers based on their browsing and purchasing history.
7. ***Cybersecurity***: FNNs are used to detect malicious activities or anomalies.
8. ***Text Classification***: FNNs are used in NLP tasks such as sentiment analysis.
9. ***Marketing***: FNNs are used to segment customers, predict customer churn

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feedback Neural Networks

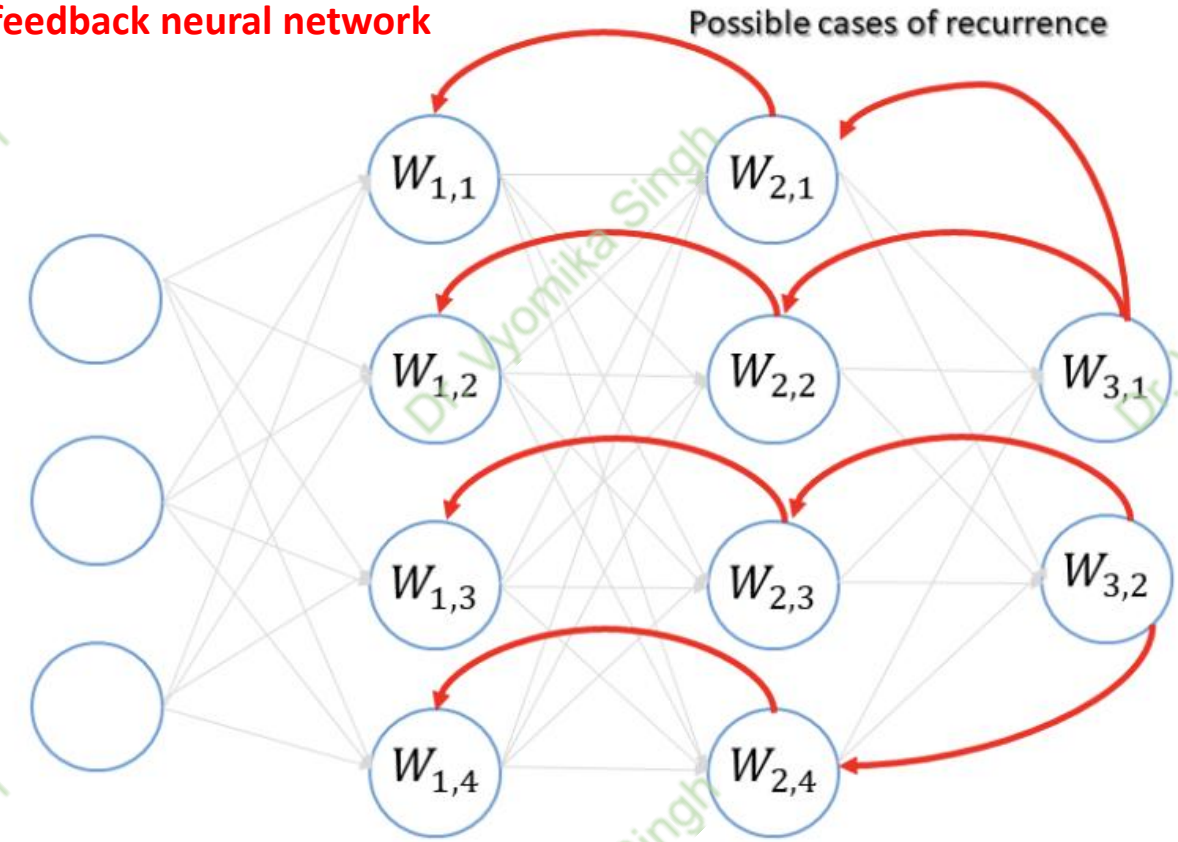
1. Feedback neural networks **do not follow any single path of transferring signals**. These kinds of networks can have signals travelling from both directions, that is, from input to output as well as from output to input.
2. Feedback neural networks are a bit complex when compared to feedforward neural networks as signals are constantly travelling from both sides.
3. These networks also possess a sense of **dynamism**. Feedback neural networks aim to attend a state of equilibrium and these networks achieve it by constantly changing themselves and by comparing the signals and units. **The state of equilibrium is maintained until there is a change in input. When the input changes, the network tries to achieve a new point of equilibrium.**

Various feedback neural network researchers have defined these networks as recurrent or interactive networks. These are generally associated with organizations that have an individual layer. The prime benefit that the feedback network model offers is that the deep neural network algorithm specifies an actual feedback system and a secondary feedback system acts as a backup to generate the result.

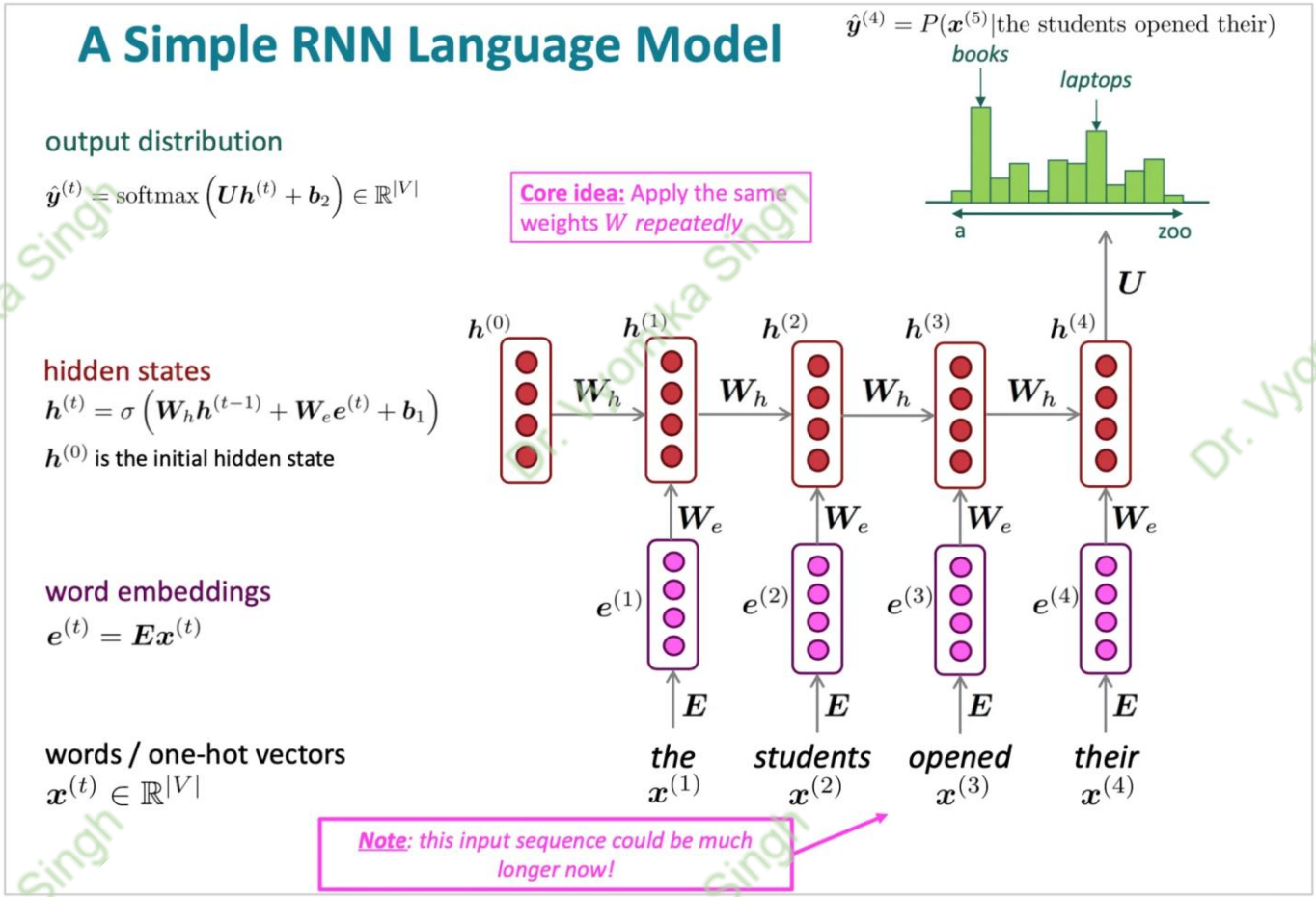
CSN 342
Deep Learning

Feedback Neural Networks

Figure: Example of a feedback neural network



CSN 342
Deep Learning



CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feedback Neural Networks

RNN Architecture: The fundamental architecture of RNN consists of the following components:

1. Input Layer: The input layer is responsible for receiving sequential data. In natural language processing tasks, for example, each element of the sequence could correspond to a word or a character

2. Hidden Layer: The hidden layer is the core component of an RNN. It maintains an internal state or hidden state that evolves as the network processes each element of the sequence. The hidden state captures information from previous time steps and, therefore, is responsible for preserving context and modeling dependencies within the sequence

3. Recurrent Connection: This is a **crucial feature** of RNNs. It is a set of weights and connections that loop back to the hidden layer from itself at the previous time step. This loop allows the RNN to maintain and update its hidden state as it processes each element in the sequence. The recurrent connection enables the network to remember and utilize information from the past.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Feedback Neural Networks

4. Output Layer: The output layer is responsible for producing predictions or outputs based on the information in the hidden state. The number of neurons in this layer depends on the specific task. For instance, in a language model, it might be a SoftMax layer to predict the next word in a sequence.

Conclusion:

RNNs are characterized by their ability to process sequential data by maintaining a hidden state that evolves over time. However, basic RNNs have limitations, such as difficulty in capturing long-range dependencies and the vanishing gradient problem. To address these issues, more advanced RNN variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), have been developed, offering improved memory and gradient flow capabilities for more effective sequence modeling.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Parameters VS Hyperparameters

Parameters:

Parameters are **internal variables** that a model learns from the training data. These are the values that the model adjusts during training to minimize the loss function and improve its predictions. Parameters are specific to the model and directly impact the model's performance.

Example: In a linear regression model, the parameters are the coefficients (weights) of the features.

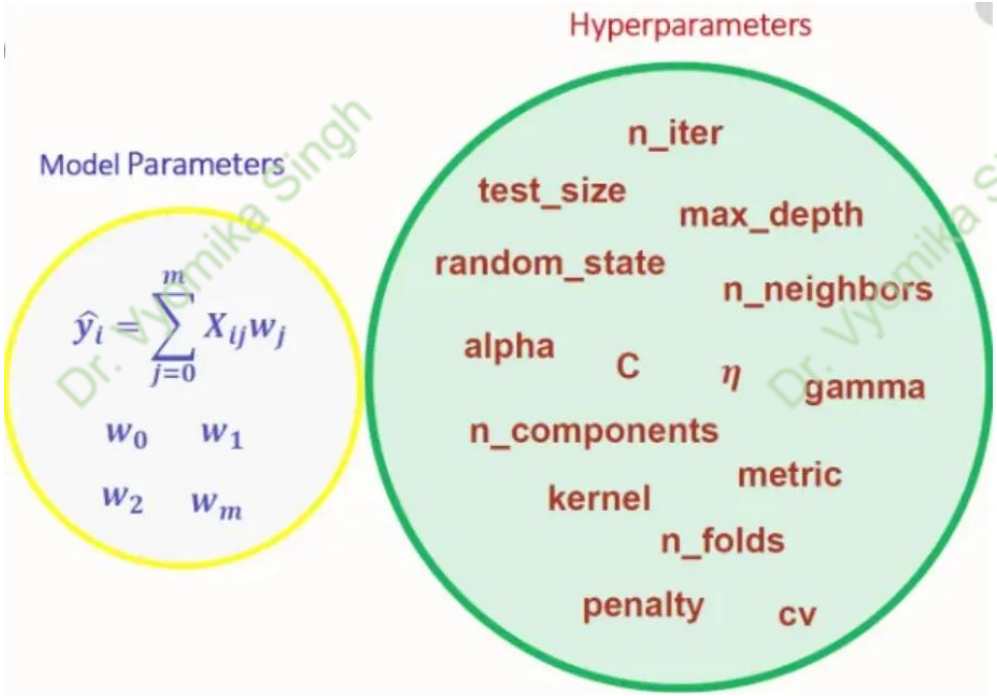
Hyperparameters:

Hyperparameters, on the other hand, **are external configurations** set before the training process begins. They are not learned from the data but are used to control the training process and the structure of the model. Proper tuning of hyperparameters is essential for optimizing model performance.

CSN 342
Deep Learning

FEATURES	PARAMETERS	HYPERPARAMETERS
Definition	Values learned by the model during training	Defined by the user to control the learning process.
Purpose	Directly impact model predictions	Control the learning process and model structure
When Set	Estimated during model training.	Set before training begins.
Tuning Method	Learned from data using optimization algorithms	Manually set or tuned using methods like grid search, random search, or Bayesian optimization
Influence on Model	Affects the output directly	Affects the speed and quality of learning
Dependence on the Dataset	Dependent	Independent
Examples	Coefficients in linear regression, weights in neural networks	Learning rate, number of layers, batch size

CSN 342
Deep Learning



Example in Practice:

Let's consider a simple neural network for image classification:

- **Parameters:** During training, the weights and biases of the neurons in the network are adjusted based on the training data. These weights and biases are the parameters.
- **Hyperparameters:** Before training, you decide on the learning rate, the number of epochs, the batch size, and the architecture of the neural network (e.g., the number of hidden layers and neurons per layer). These are the hyperparameters.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Learning Parameters

In ML, learning parameters are numbers inside the model (like weights) that we adjust so the model performs better.

Parameters = things the model learns.

Let's say we have a model:

$\hat{y} = f(x;\theta)$

where:

- x = input
- $\hat{y}$ = prediction
- θ = **learning parameters** (weights, biases, etc.)

Learning Parameters:

These are:

- Weights w
- Biases b
- Any trainable numbers in the network

Example (Linear Regression):

$\hat{y} = wx + b$

Here:

- w, b are **learning parameters**
- Training = finding best w, b

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Loss Functions

Loss function measures **error for one data point**.

Mean Squared Error (MSE):

L(y, y^)=(y−y^) power raised to 2

Cross Entropy (for classification)

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=0}^{N}(y_i - \hat{y}_i)^2$$

y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\hat{\vec{y}}$

Absolute Error:

L(y, y^)=|y−y^|

$$\mathcal{L}(\theta) = -\sum_{i=0}^{N}\hat{y}_i \cdot log(y_i)$$

y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\hat{\vec{y}}$

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions

Without activation functions, neural networks behave like **linear regression**, **regardless of depth**. Nonlinearity enables NNs to approximate complex functions (Universal Approximation Theorem). Activation functions are crucial mathematical functions in neural networks that introduce non-linearity, allowing models to learn complex patterns beyond simple linear relationships, **deciding whether a neuron should "fire" (activate) based on its weighted inputs**, with common types including ReLU, Sigmoid, and Tanh, essential for enabling deep learning. Without them, even deep networks would act as simple linear models, limiting their power.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions

Standard definition: An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.

Three types of Activation Functions:

1. Binary Step Function
2. Linear Activation Function
3. Non-Linear Activation Function

CSN 342
Deep Learning

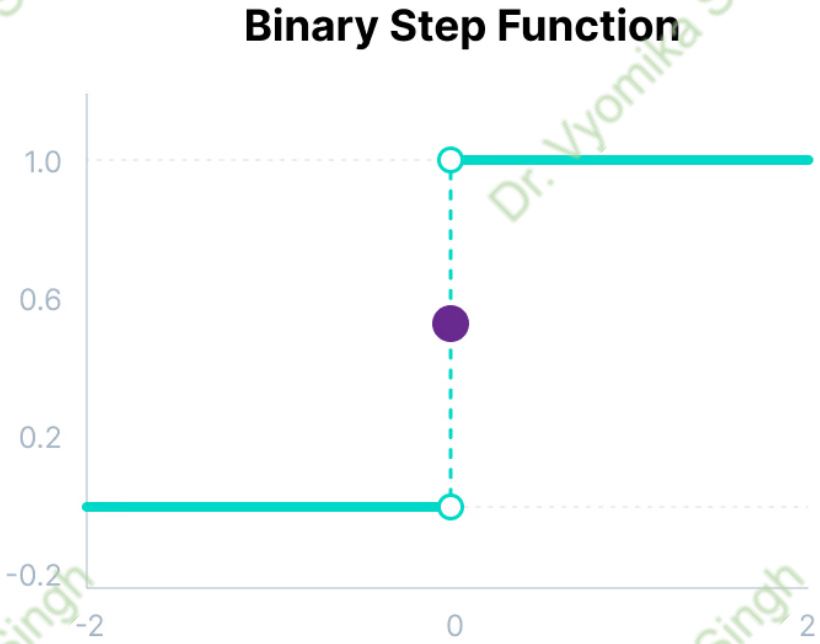
DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions: Binary Step Function

Binary step function depends on a threshold value that decides whether a neuron should be activated or not. The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.

Binary step

$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$



CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions: Binary Step Function

Limitations of binary step function:

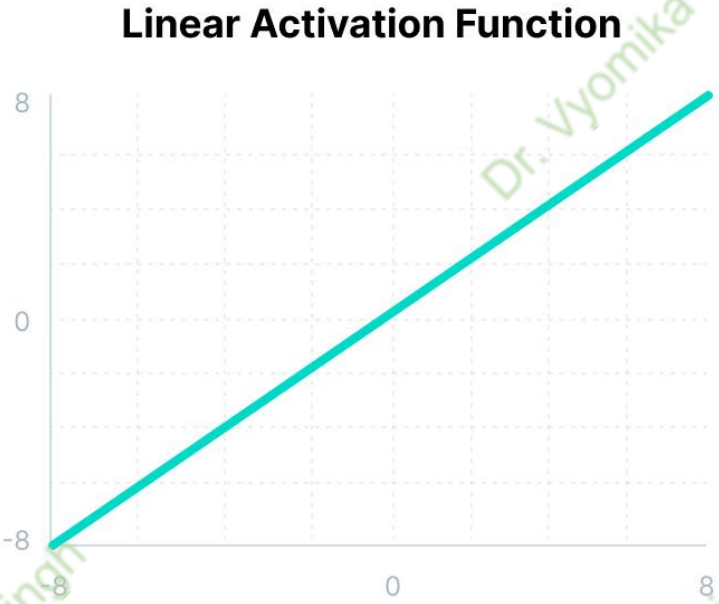
1. It cannot provide multi-value outputs; for example, it cannot be used for multi-class classification problems.
2. The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions: Linear Activation Function

The linear activation function, also known as **"no activation"** or **"identity function"** (multiplied x 1.0), is where the activation is proportional to the input. The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.



Linear

$$f(x) = x$$

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions: Linear Activation Function

However, a linear activation function has two major problems :

1. It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x.
2. All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

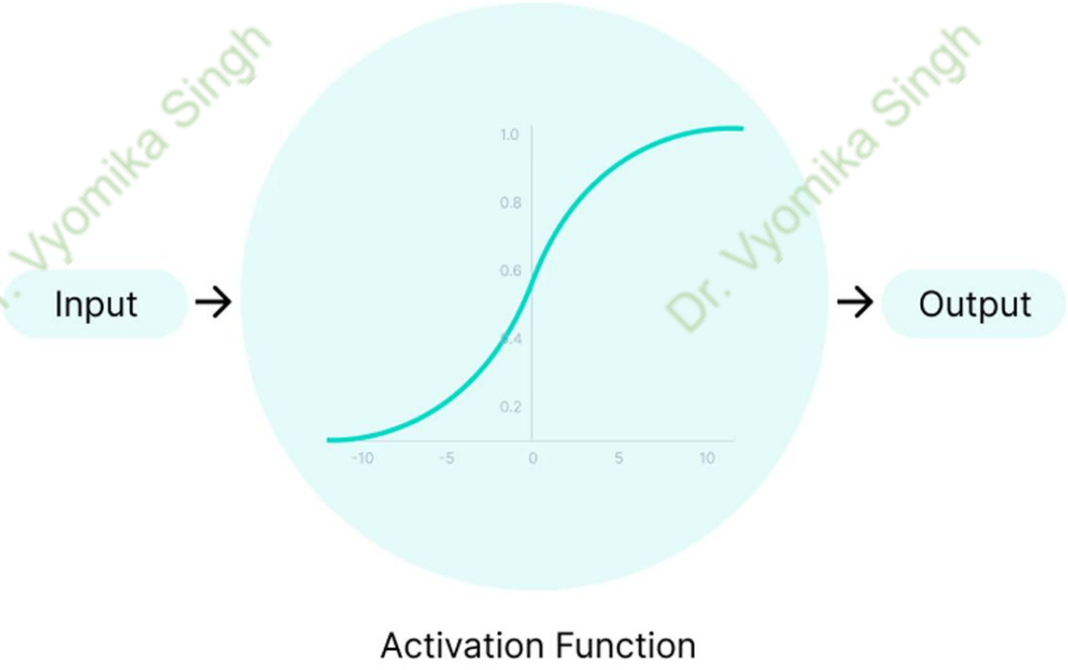
Activation functions: Non-Linear Activation Function

A non-linear **activation function** applies a **non-linear transformation; Non-linearity breaks the linearity trap**.

Modern neural networks learn using:
1. Gradient Descent
2. Backpropagation

That requires: df/dz to exist

So we need activation functions that are:
1. Non-linear
2. Differentiable (almost everywhere)



Activation Function

CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

Activation functions: Non-Linear Activation Function

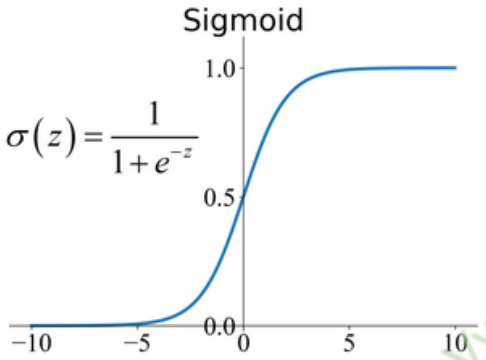
Non-linear activation functions solve the following limitations of linear activation functions:

1. They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
2. They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

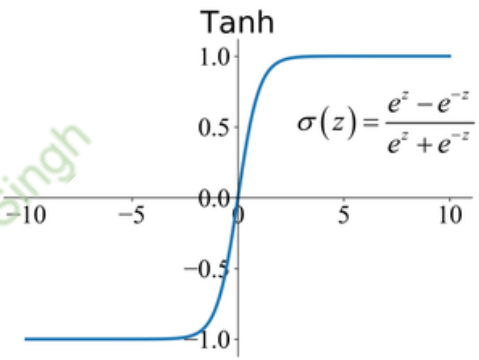
CSN 342
Deep Learning

DIT UNIVERSITY
IMAGINE ASPIRE ACHIEVE

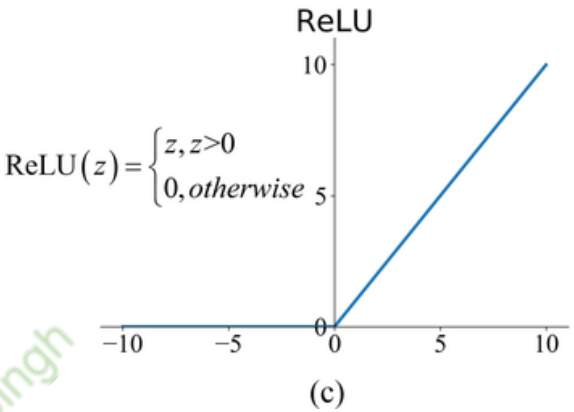
Common Activation functions



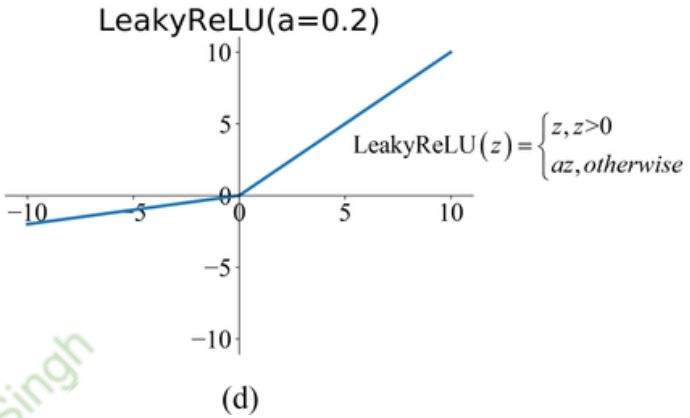
(a)



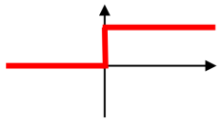
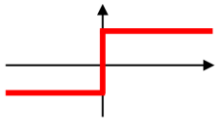
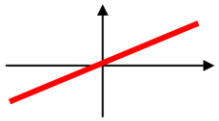
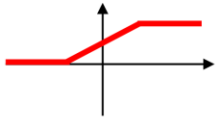
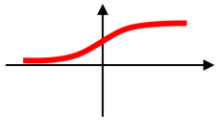
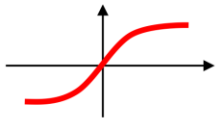
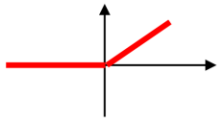
(b)



(c)



(d)

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^{z})$	Multi-layer Neural Networks